ПРАКТИЧНЕ ЗАНЯТТЯ №4

Питання для самостійного опрацювання

з дисципліни: «Об’єктно-орієнтоване програмування»

Тема: Успадкування, поліморфізм, віртуальні функції та абстрактні класи

Курсова робота: «Гра П’ятнашки»

Виконав: Вівчар Вадим Вікторович

Група: АЛК-43

Мета роботи

Узагальнити теоретичні положення щодо повторного використання коду, успадкування, поліморфізму, віртуальних функцій та абстрактних класів і показати їх застосування в контексті курсової роботи «Гра П’ятнашки».

Теоретичні пояснення та приклади

1. Що називається повторним використанням коду?

Повторне використання коду — це застосування вже створених класів, методів і компонентів у нових частинах програми без повторного написання однакової логіки. Такий підхід зменшує дублювання, спрощує супровід і прискорює розробку.

Приклад для курсової роботи «Гра П’ятнашки».
У курсовій роботі «Гра П’ятнашки» спільну логіку відображення елемента можна винести в базовий клас PuzzleElement, а потім повторно використовувати її в класах NumberTile і EmptyCell.

2. Який механізм дозволяє створювати ієрархії класів?

Створювати ієрархії класів дозволяє механізм успадкування. Він дає можливість будувати нові класи на основі вже існуючих, зберігаючи спільні властивості та додаючи спеціалізовану поведінку.

Приклад для курсової роботи «Гра П’ятнашки».
У грі «П’ятнашки» від класу PuzzleElement можна успадкувати класи NumberTile, EmptyCell і AnimatedTile.

3. Який клас називається базовим?

Базовим називається клас, від якого успадковуються інші класи. У ньому зазвичай описують спільні дані та методи, характерні для всіх похідних об’єктів.

Приклад для курсової роботи «Гра П’ятнашки».
Для курсової базовим може бути клас PuzzleElement, який містить координати, розмір і загальний інтерфейс відображення.

4. Який клас називається похідним?

Похідним називається клас, що створюється на основі базового і успадковує його властивості та методи. Похідний клас може додавати власні поля, методи або перевизначати успадковану поведінку.

Приклад для курсової роботи «Гра П’ятнашки».
Наприклад, NumberTile є похідним від PuzzleElement і додатково зберігає число на плитці.

5. Що називається виводом класу?

Виводом класу називають створення нового класу на основі вже існуючого базового класу за допомогою успадкування. У результаті формується новий тип, який зберігає спільні риси базового класу і розширює їх.

Приклад для курсової роботи «Гра П’ятнашки».
У проєкті «П’ятнашки» виводом класу буде створення класу EmptyCell на основі PuzzleElement.

6. Назвіть види успадкування залежно від кількості базових класів.

За кількістю базових класів розрізняють одиночне та множинне успадкування. В одиночному похідний клас має один базовий клас, а в множинному — два або більше.

Приклад для курсової роботи «Гра П’ятнашки».
Приклад одиночного успадкування: NumberTile успадковується від PuzzleElement. Приклад множинного: AnimatedTile може успадковуватися від PuzzleElement і Drawable.

7. Опишіть механізм успадкування.

Механізм успадкування полягає в тому, що похідний клас автоматично отримує доступні члени базового класу і може використовувати їх у власній логіці. Це дозволяє будувати програми як ієрархію пов’язаних типів із поступовим уточненням властивостей.

Приклад для курсової роботи «Гра П’ятнашки».
У «П’ятнашках» похідні плитки отримують спільні координати й метод малювання від базового елемента поля.

8. Назвіть види успадкування залежно від рівня доступу.

Залежно від рівня доступу розрізняють відкрите, захищене та закрите успадкування. Вони визначають, як саме члени базового класу будуть доступні в похідному класі.

Приклад для курсової роботи «Гра П’ятнашки».
У курсовій найдоречнішим є відкрите успадкування, коли PuzzleElement є загальним типом для всіх елементів поля.

9. Опишіть відкрите успадкування.

Відкрите успадкування задається ключовим словом public. При цьому відкриті члени базового класу залишаються відкритими, а захищені — захищеними в похідному класі.

Приклад для курсової роботи «Гра П’ятнашки».
Якщо NumberTile успадковується від PuzzleElement як public, то об’єкт NumberTile можна розглядати як PuzzleElement.

10. Опишіть захищене успадкування.

Захищене успадкування задається словом protected. У цьому випадку відкриті й захищені члени базового класу стають захищеними в похідному класі.

Приклад для курсової роботи «Гра П’ятнашки».
Такий варіант можна застосувати для службових класів, які не повинні відкривати інтерфейс базового класу зовнішньому коду гри.

11. Опишіть закрите успадкування.

Закрите успадкування задається словом private. Відкриті та захищені члени базового класу стають закритими в похідному класі і не доступні ззовні через інтерфейс похідного класу.

Приклад для курсової роботи «Гра П’ятнашки».
У «П’ятнашках» закрите успадкування могло б використовуватись для внутрішнього допоміжного класу, але для основної ієрархії елементів поля воно менш зручне.

12. Як викликаються конструктори і деструктори в ієрархії класів?

Під час створення об’єкта спочатку викликаються конструктори базових класів, а потім конструктор похідного класу. Під час знищення порядок зворотний: спочатку деструктор похідного класу, потім деструктор базового.

Приклад для курсової роботи «Гра П’ятнашки».
Коли створюється об’єкт NumberTile, спершу ініціалізується PuzzleElement, а вже потім сама плитка.

13. Опишіть оператор dynamic_cast.

Оператор dynamic_cast використовується для безпечного перетворення в ієрархії поліморфних класів. Він дозволяє перевірити реальний тип об’єкта під час виконання програми.

Приклад для курсової роботи «Гра П’ятнашки».
Якщо покажчик PuzzleElement* може вказувати на NumberTile або EmptyCell, то через dynamic_cast можна перевірити, чи є об’єкт саме числовою плиткою.

14. Які базові класи називаються віртуальними?

Віртуальними називаються базові класи, які успадковуються з ключовим словом virtual, щоб уникнути дублювання спільного базового підоб’єкта при множинному успадкуванні. Це особливо важливо в так званій ромбоподібній ієрархії.

Приклад для курсової роботи «Гра П’ятнашки».
Якщо класи Drawable і Movable успадковуються від GameObject, а AnimatedTile успадковується від обох, то GameObject доцільно зробити віртуальним базовим класом.

15. Чим динамічний поліморфізм відрізняється від статичного?

Статичний поліморфізм визначається на етапі компіляції, наприклад через перевантаження функцій або шаблони. Динамічний поліморфізм працює під час виконання програми і базується на віртуальних функціях.

Приклад для курсової роботи «Гра П’ятнашки».
У «П’ятнашках» перевантаження методу move() для різних параметрів є прикладом статичного поліморфізму, а виклик draw() через покажчик на PuzzleElement — динамічного.

16. Яка функція називається віртуальною?

Віртуальною називається функція базового класу, оголошена з ключовим словом virtual, для якої можна виконувати перевизначення в похідних класах. Під час виклику через покажчик або посилання на базовий клас буде обрано реалізацію реального типу об’єкта.

Приклад для курсової роботи «Гра П’ятнашки».
У базовому класі PuzzleElement віртуальною може бути функція draw(), а класи NumberTile та EmptyCell реалізують її по-своєму.

17. Що таке заміщення?

Заміщенням називається перевизначення віртуальної функції базового класу в похідному класі. Саме завдяки цьому кожний похідний клас може надавати власну реалізацію спільного інтерфейсу.

Приклад для курсової роботи «Гра П’ятнашки».

У грі «П’ятнашки» NumberTile може заміщувати метод draw(), щоб відображати число, а EmptyCell — порожню клітинку.

18. Які обмеження накладаються на віртуальні функції?

Віртуальні функції не можуть бути static, оскільки вони пов’язані з конкретним об’єктом. Конструктори не можуть бути віртуальними, а під час їх виконання механізм динамічного поліморфізму працює обмежено.

Приклад для курсової роботи «Гра П’ятнашки».
У курсовій віртуальною доцільно робити draw() або update(), але не конструктор PuzzleElement.

19. Що таке поліморфічний кластер?

Поліморфічним кластером називають групу об’єктів різних похідних класів, з якими працюють через спільний базовий тип. Це дозволяє однаково обробляти різні об’єкти, викликаючи спільний інтерфейс.

Приклад для курсової роботи «Гра П’ятнашки».
Для «П’ятнашок» поліморфічним кластером може бути контейнер з покажчиками на PuzzleElement, у якому зберігаються NumberTile та EmptyCell.

20. Що станеться, якщо віртуальна функція не заміщується у якомусь похідному класі?

Якщо похідний клас не перевизначає звичайну віртуальну функцію, то використовується реалізація базового класу. Якщо ж базова функція є суто віртуальною, то такий похідний клас також залишається абстрактним.

Приклад для курсової роботи «Гра П’ятнашки».
Якщо EmptyCell не замістить звичайну віртуальну функцію draw(), то буде викликано базову версію PuzzleElement::draw().

21. Які класи називаються абстрактними?

Абстрактними називаються класи, які містять хоча б одну суто віртуальну функцію і не можуть створюватися як окремі об’єкти. Вони призначені для задання спільного інтерфейсу для похідних класів.

Приклад для курсової роботи «Гра П’ятнашки».
У грі «П’ятнашки» клас PuzzleElement можна зробити абстрактним, щоб створювати лише конкретні типи елементів, а не узагальнений елемент.

22. Яка функція називається суто віртуальною?

Суто віртуальною називається віртуальна функція, яка оголошується з записом = 0. Вона визначає обов’язковий інтерфейс, який має бути реалізований у конкретних похідних класах.

Приклад для курсової роботи «Гра П’ятнашки».
Наприклад, метод draw() у PuzzleElement можна оголосити суто віртуальним, щоб кожна плитка обов’язково мала власну реалізацію відображення.

23. Що таке таблиця віртуальних функцій?

Таблиця віртуальних функцій, або vtable, — це внутрішній механізм, який використовується компілятором для підтримки динамічного поліморфізму. Через неї під час виконання визначається, яку саме реалізацію віртуальної функції треба викликати.

Приклад для курсової роботи «Гра П’ятнашки».
Коли масив покажчиків на PuzzleElement викликає draw() для різних плиток, правильна версія методу обирається саме через таблицю віртуальних функцій.

24. Коли застосовується віртуальний деструктор?

Віртуальний деструктор застосовують тоді, коли об’єкти похідних класів планується знищувати через покажчик на базовий клас. Це гарантує правильний виклик усіх деструкторів у ієрархії.

Приклад для курсової роботи «Гра П’ятнашки».
Якщо елементи поля «П’ятнашок» зберігаються як PuzzleElement* і видаляються через цей покажчик, деструктор базового класу обов’язково має бути віртуальним.

Висновок

У ході виконання практичного заняття було систематизовано основні поняття об’єктно-орієнтованого програмування, пов’язані з повторним використанням коду, успадкуванням і поліморфізмом. Наведені приклади показують, що для курсової роботи «Гра П’ятнашки» доцільно використовувати базовий клас для елементів поля, похідні класи для конкретних типів плиток, а також віртуальні функції для відображення, оновлення стану та безпечного видалення об’єктів. Такий підхід робить проєкт структурованим, розширюваним і зручним для супроводу.

Приклад 1. Базовий і похідний класи. Відкрите успадкування та віртуальна функція

Microsoft Visual Studio

```
#include <iostream>
using namespace std;

class PuzzleElement {
protected:
    int x, y;

public:
    PuzzleElement(int x, int y) : x(x), y(y) {
        cout << "PuzzleElement constructor" << endl;
    }

    virtual void show() const {
        cout << "Base element at (" << x << ", " << y << ")" << endl;
    }

    virtual ~PuzzleElement() {
        cout << "PuzzleElement destructor" << endl;
    }
};

class NumberTile : public PuzzleElement {
private:
    int value;

public:
    NumberTile(int x, int y, int value) : PuzzleElement(x, y), value(value) {
        cout << "NumberTile constructor" << endl;
    }

    void show() const override {
```

Рисунок 1 – Скрін коду прикладу 1 у Visual Studio

Microsoft Visual Studio Debug Console

```
PuzzleElement constructor
NumberTile constructor
Tile 5 at (1, 2)
NumberTile destructor
PuzzleElement destructor

E:\user vadim\Don\Tema4\x64\Debug\Tema4.exe exited with code 0 (0x0).
Press any key to close this window . . .
```

Рисунок 2 – Скрін результату виконання прикладу 1

Приклад 2. Абстрактний клас, суто віртуальна функція та dynamic_cast

Microsoft Visual Studio

```
#include <iostream>
using namespace std;

class PuzzleObject {
public:
    virtual void draw() const = 0;

    virtual ~PuzzleObject() {
        cout << "PuzzleObject destructor" << endl;
    }
};

class NumberTile : public PuzzleObject {
private:
    int value;

public:
    NumberTile(int value) : value(value) {
        cout << "NumberTile constructor" << endl;
    }

    void draw() const override {
        cout << "Drawing number tile: " << value << endl;
    }

    void showValue() const {
        cout << "Tile value = " << value << endl;
    }

    ~NumberTile() {
        cout << "NumberTile destructor" << endl;
```

Рисунок 3 – Скрін коду прикладу 2 у Visual Studio

Microsoft Visual Studio Debug Console

```
NumberTile constructor
EmptyCell constructor
Drawing number tile: 8
Drawing empty cell
Tile value = 8
obj2 is not a NumberTile
NumberTile destructor
PuzzleObject destructor
EmptyCell destructor
PuzzleObject destructor

E:\user vadim\Don\Tema4.1\x64\Debug\Tema4.1.exe exited with code 0 (0x0).
Press any key to close this window . . .
```

Рисунок 4 – Скрін результату виконання прикладу 2

Приклад 3. Protected і private успадкування. Порядок виклику конструкторів і деструкторів

Microsoft Visual Studio

```
#include <iostream>
using namespace std;

class Base {
public:
    Base() {
        cout << "Base constructor" << endl;
    }

    void publicMethod() const {
        cout << "Base public method" << endl;
    }

protected:
    void protectedMethod() const {
        cout << "Base protected method" << endl;
    }

public:
    ~Base() {
        cout << "Base destructor" << endl;
    }
};

class ProtectedDerived : protected Base {
public:
    ProtectedDerived() {
        cout << "ProtectedDerived constructor" << endl;
    }

    void test() {
```

Рисунок 5 – Скрін коду прикладу 3 у Visual Studio

Microsoft Visual Studio Debug Console

```
Protected inheritance:
Base constructor
ProtectedDerived constructor
Base public method
Base protected method

Private inheritance:
Base constructor
PrivateDerived constructor
Base public method
Base protected method
PrivateDerived destructor
Base destructor
ProtectedDerived destructor
Base destructor

E:\user vadim\Don\Tema4.2\x64\Debug\Tema4.2.exe exited with code 0 (0x0).
Press any key to close this window . . .
```

Рисунок 6 – Скрін результату виконання прикладу 3

Приклад 4. Множинне успадкування та віртуальний базовий клас

Microsoft Visual Studio

```
#include <iostream>
using namespace std;

class GameObject {
public:
    GameObject() {
        cout << "GameObject constructor" << endl;
    }

    void info() const {
        cout << "Shared base class GameObject" << endl;
    }

    ~GameObject() {
        cout << "GameObject destructor" << endl;
    }
};

class Drawable : virtual public GameObject {
public:
    Drawable() {
        cout << "Drawable constructor" << endl;
    }

    ~Drawable() {
        cout << "Drawable destructor" << endl;
    }
};

class Movable : virtual public GameObject {
public:
```

Рисунок 7 – Скрін коду прикладу 4 у Visual Studio

Microsoft Visual Studio Debug Console

```
GameObject constructor
Drawable constructor
Movable constructor
Tile constructor
Shared base class GameObject
Tile uses one shared GameObject base
Tile destructor
Movable destructor
Drawable destructor
GameObject destructor

E:\user vadim\Don\Tema4.4\x64\Debug\Tema4.4.exe exited with code 0 (0x0).
Press any key to close this window . . .
```

Рисунок 8 – Скрін результату виконання прикладу 4